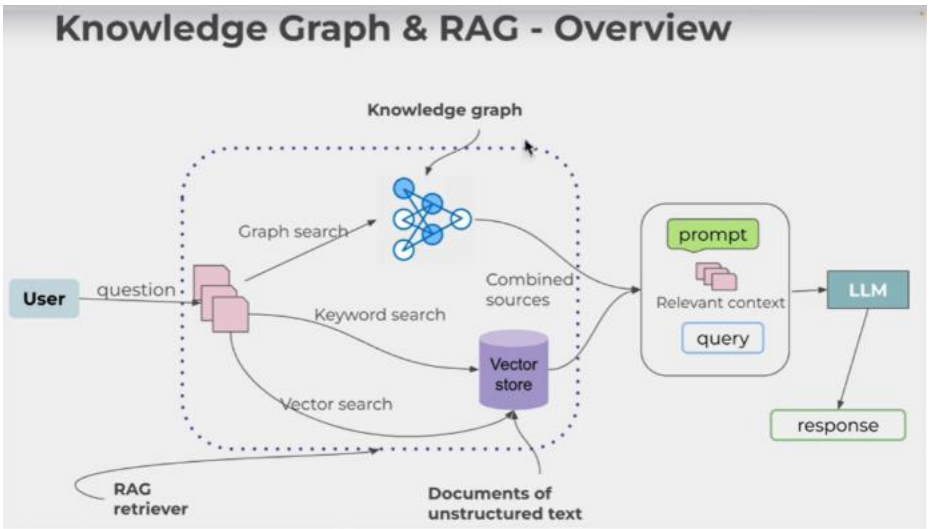
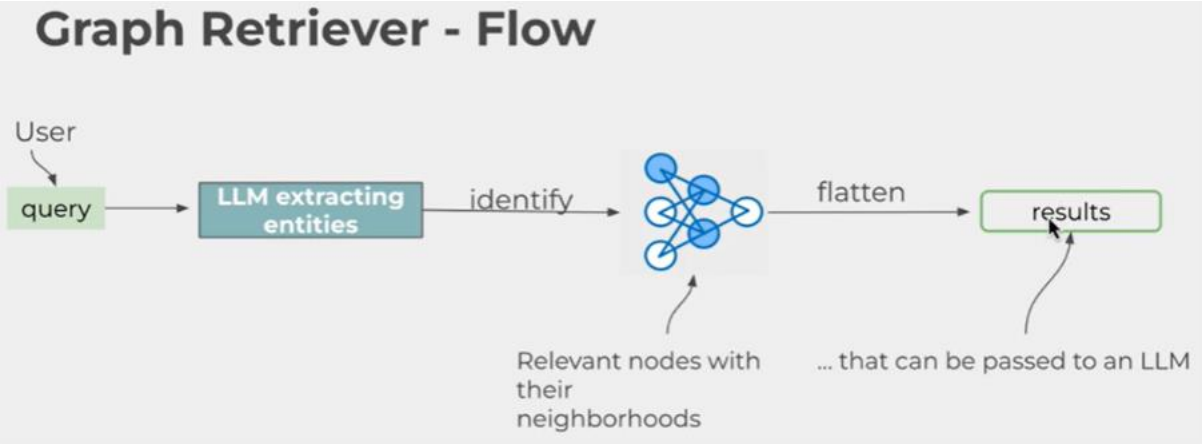
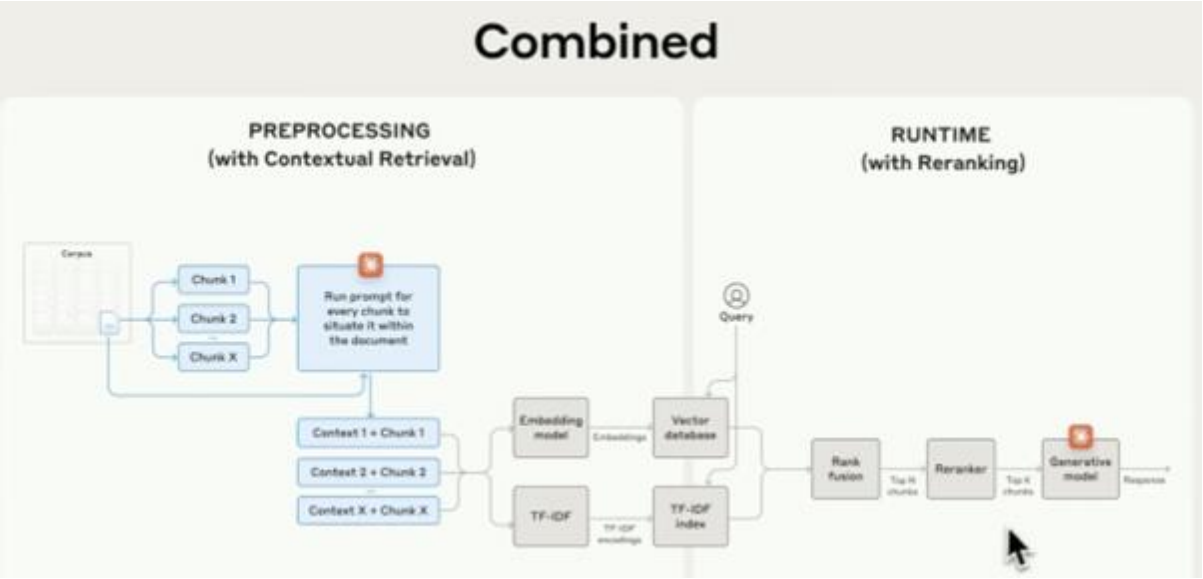
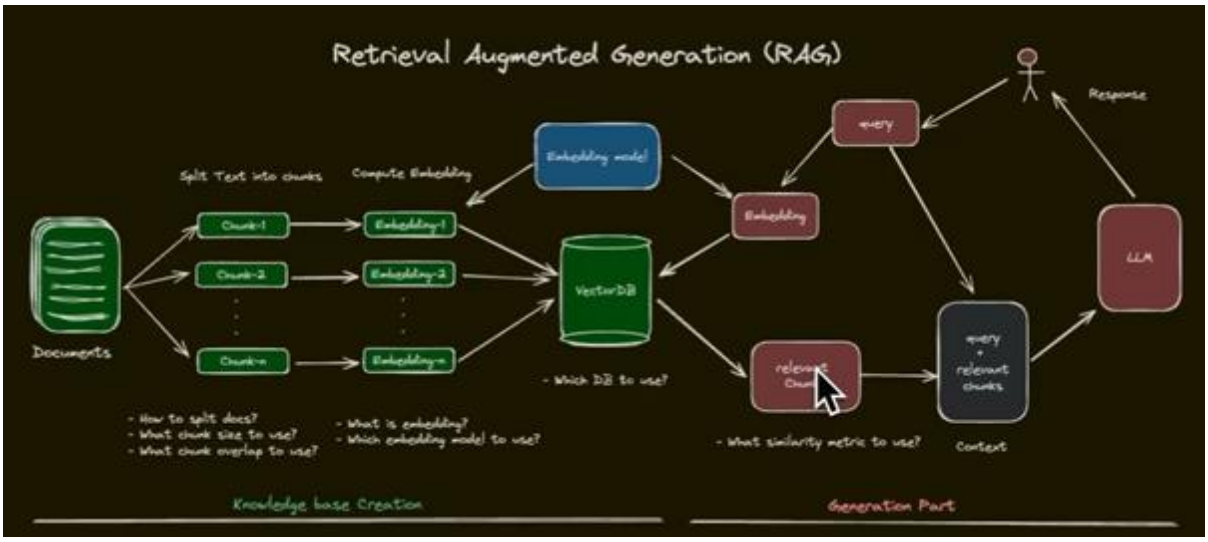
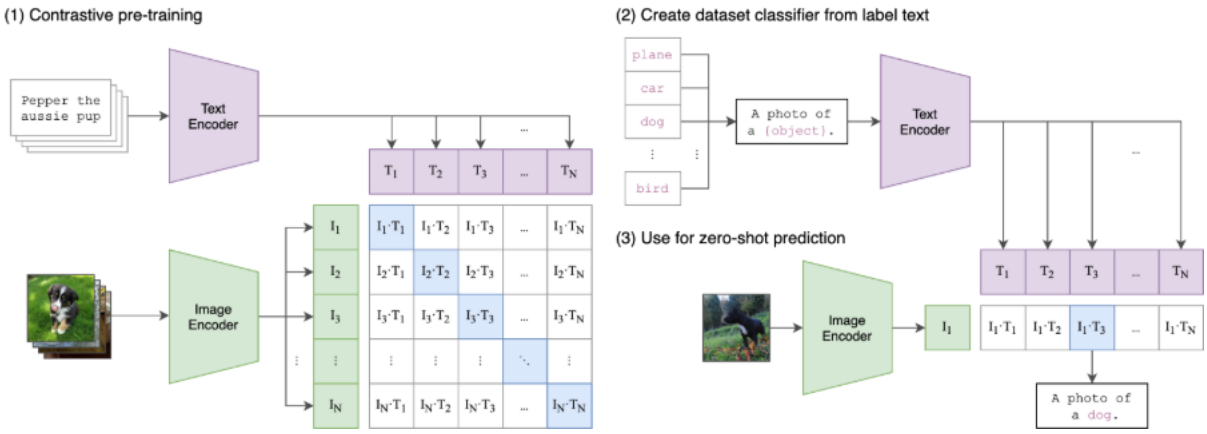

Knowledge Graph & RAG - Overview
Knowledge graph
Graph search
User
question
Keyword search
Combined sources
Vector store
Vector search
prompt
Relevant context
query
LLM
response
RAG retriever
Documents of unstructured text


Graph Retriever - Flow
User
query
LLM extracting entities
identify
flatten
results
Relevant nodes with their neighborhoods
... that can be passed to an LLM


Combined
PREPROCESSING
(with Contextual Retrieval)
RUNTIME
(with Reranking)
Chunk 1
Chunk 2
Chunk X
Run prompt for every chunk to situate it within the document
Context 1 + Chunk 1
Context 2 + Chunk 2
Context X + Chunk X
Embedding model
Vector database
TF-IDF
TF-IDF index
Query
Rank fusion
Reranker
Generative model





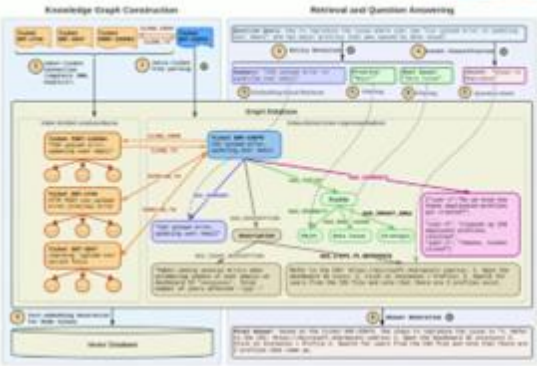
Strategy	Best Performance On...	CPU Impact	Overall Verdict
BM25	Exact codes, acronyms, and technical IDs.	Negligible	Essential safety net.
Semantic	Broad questions, "How-to" intent.	Medium	Good, but needs reranking.
Hybrid	Multi-topic PDFs and complex queries.	Medium	**Winner** for enterprise docs.
MMR	Long PDFs with repetitive info.	Low	Best for summarization tasks.

① Higher Accuracy

The clearest driver of GraphRAG adoption amongst users is *higher accuracy.*

"Across all metrics, our method demonstrates consistent improvements. ***Notably, it surpasses the baseline by 77.6% in MRR and by 0.32 in BLEU score,*** *substantiating its superior retrieval efficacy and question-answering accuracy."*

 Neo4j Inc. All rights reserved 2023



① Higher Accuracy

The clearest driver of GraphRAG adoption amongst users is *higher accuracy.*

 Neo4j Inc. All rights reserved 2023

Microsoft Research Blog

GraphRAG: Unlocking LLM discovery on narrative private data

Published February 15, 2024

"By combining LLM-generated knowledge graphs and graph machine learning, **GraphRAG enables us to answer important classes of questions** that we cannot attempt with baseline RAG alone."

Microsoft

GraphRAG Retrieval Patterns

The core GraphRAG pattern is very simple, yet powerful.

1. Do a vector search to find an initial set of nodes
2. Traverse the graph around those nodes to add context
3. (*Optional*: Rank the results using the graph and pass the *top-k* documents to the LLM)

Test Case	BM25 (Keyword)	Semantic (Vector)	Hybrid (Combined)
Specific Code (e.g., "Error 503")	**Excellent.** Finds exact matches.	**Poor.** Might match "Error" in general.	**Best.** Keeps the exact match at the top.
Synonyms (e.g., "login assistance")	**Poor.** Won't find "authentication guide."	**Excellent.** Understands they mean the same.	**Balanced.** Finds the synonym and the exact word.
Short Queries (e.g., "passcode")	**Strong.** Finds every instance.	**Vague.** Might find too much context.	**Strong.** Filters for relevance.
Long Questions	**Noisy.** Gets distracted by words like 'the'.	**Strong.** Focuses on the core "intent."	**Best.** Filters noise while keeping keywords.

https://platform.openai.com/docs/guides/structured-outputs

https://huggingface.co/ibm-granite/granite-vision-3.2-2b

https://developer.ibm.com/articles/awb-enhancing-rag-performance-chunking-strategies/

https://www.datacamp.com/tutorial/multimodal-rag

https://developer.nvidia.com/blog/an-easy-introduction-to-multimodal-retrieval-augmented-generation/

https://console.cloud.google.com/vertex-ai/publishers/google/model-garden/medgemma

https://huggingface.co/google/medgemma-1.5-4b-it

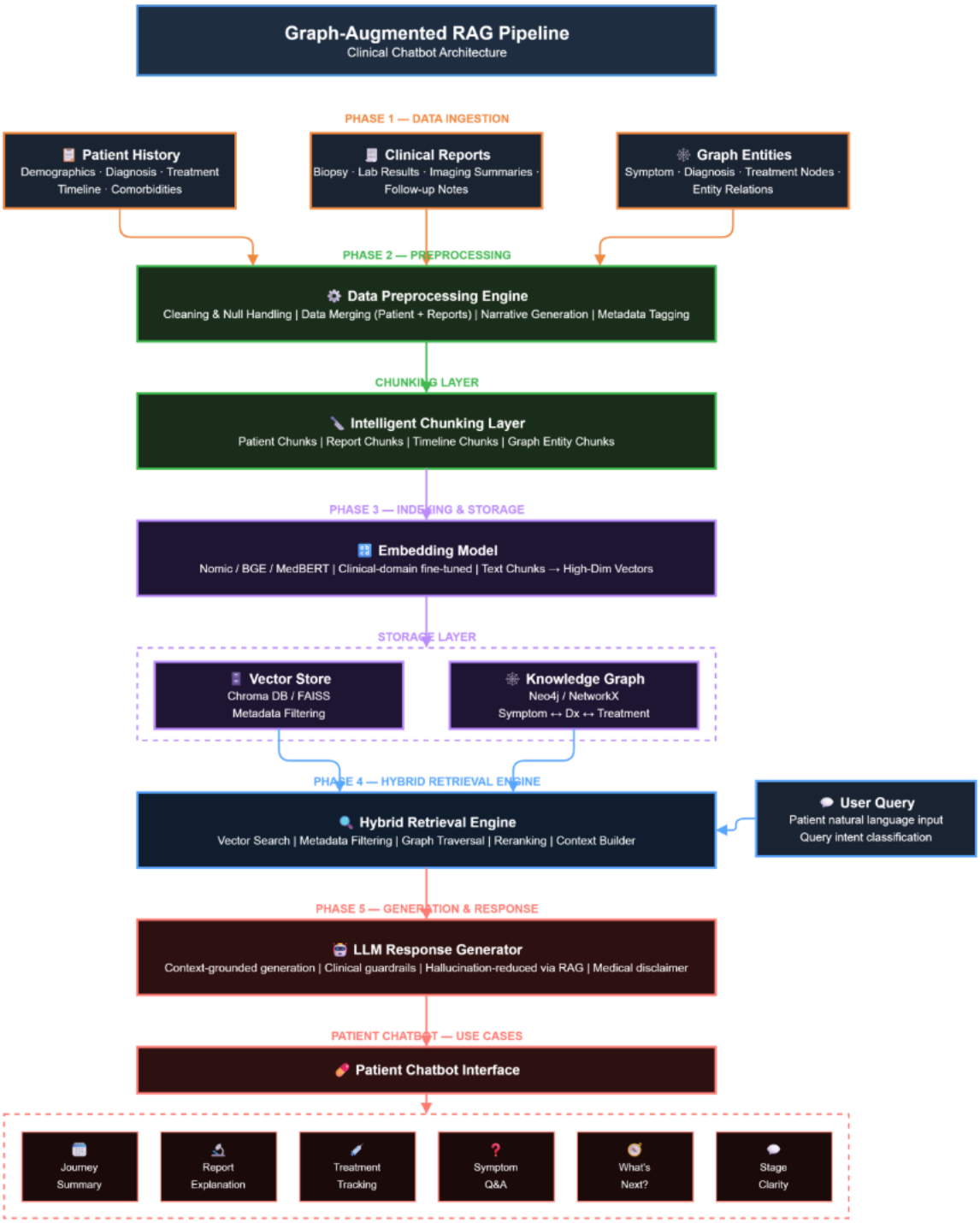
https://syntheticheath.github.io/synthea/

```
Cancer
  ├─[:TREATED_WITH]──────────► ChemoDrug
  │                                  │
  ├─[:HAS_BIOMARKER]─────► Biomarker     ├─[:CAUSES_EATING_EFFECT]──► EatingAdverseEffect
  │                                  │                                          │
  └─[:TREATED_BY_PROTOCOL]► TreatmentProtocol                   ├─[:WORSENED_BY]────► FoodItem
        │                        │                                ├─[:RELIEVED_BY]────► FoodItem
        └─[:INCLUDES_DRUG]────►┘                                  └─[:MANAGED_BY]─────► NutritionGuideline
                                 │
                         ├─[:MAY_CAUSE]──────────────────► SideEffect
                         │                                     │
                         └─[:DESCRIBED_BY]────► DrugInteraction    └─[:LEADS_TO_EATING_EFFECT]──► EatingAdverseEffect
                                                   ▲
NonChemoDrug ────[:HAS_INTERACTION_WITH]──► ChemoDrug
    │
    └─[:TREATS]──────────────────────► Ailment
```

EnsembleRetriever | langchain_classic | LangChain Reference

Evaluating Medical RAG with NVIDIA AI Endpoints and Ragas | NVIDIA Technical Blog

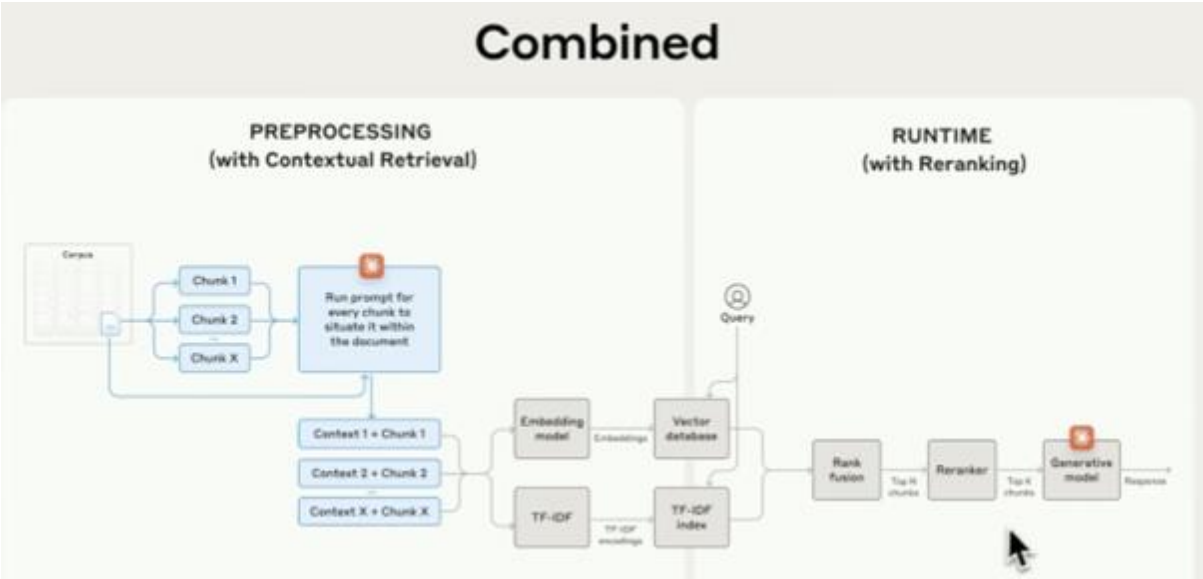
MACCROBAT


Graph-Augmented RAG Pipeline
Clinical Chatbot Architecture
PHASE 1 — DATA INGESTION
Patient History
Demographics · Diagnosis · Treatment Timeline · Comorbidities
Clinical Reports
Biopsy · Lab Results · Imaging Summaries · Follow-up Notes
Graph Entities
Symptom · Diagnosis · Treatment Nodes · Entity Relations
PHASE 2 — PREPROCESSING
Data Preprocessing Engine
Cleaning & Null Handling | Data Merging (Patient + Reports) | Narrative Generation | Metadata Tagging
CHUNKING LAYER
Intelligent Chunking Layer
Patient Chunks | Report Chunks | Timeline Chunks | Graph Entity Chunks
PHASE 3 — INDEXING & STORAGE
Embedding Model
Nomic / BGE / MedBERT | Clinical-domain fine-tuned | Text Chunks → High-Dim Vectors
STORAGE LAYER
Vector Store
Chroma DB / FAISS
Metadata Filtering
Knowledge Graph
Neo4j / NetworkX
Symptom ↔ Dx ↔ Treatment
PHASE 4 — HYBRID RETRIEVAL ENGINE
Hybrid Retrieval Engine
Vector Search | Metadata Filtering | Graph Traversal | Reranking | Context Builder
User Query
Patient natural language input
Query intent classification
PHASE 5 — GENERATION & RESPONSE
LLM Response Generator
Context-grounded generation | Clinical guardrails | Hallucination-reduced via RAG | Medical disclaimer
PATIENT CHATBOT — USE CASES
Patient Chatbot Interface
Journey Summary
Report Explanation
Treatment Tracking
Symptom Q&A
What's Next?
Stage Clarity

Current Status of Local Assets

Your uploaded files are now being used to define the graph schema:

- **`patients.csv`**: Providing the ground truth for **stages** and treatment plans *(MAP chemotherapy)*.
- **`Insilco - reports_index.csv`**: Acting as the lookup table to link specific MRI/CT/X-ray text files to session caching (browser + user compliances)
-



Based on the pipeline established on Tuesday, the system follows a **Three-Tier Architecture**:

1. **Ingestion Tier:** Processes the **Figshare Radiograph dataset** and **TCGA-SARC MRI** metadata.
2. **Embedding Tier:** Uses LLM - OpenAI Vision to interpret visual features (like the *sunburst periosteal reaction* in **Osteosarcoma**) and converts them into vectors.
3. **Knowledge Tier:** Maps these vectors to the Neo4j graph, where "Osteosarcoma" nodes are linked to "Complications" such as **Avascular Necrosis (AVN)** or **Osteonecrosis**, allowing the RAG to explain *why* a certain treatment leads to bone death.

Week 1

Overview This week focused on completing administrative onboarding, understanding the core project architecture ("Multi-modal Agentic RAG"), and setting up the local development environment. Additionally, I contributed to the company-wide AI upskilling initiative.

1. Administrative & Onboarding

- **Onboarding:** Completed all documentation and form submissions (Jan 19).
- **Induction:** Attended team induction with Suraj Sir; received clarity on the three core objectives for the internship.

2. Project: Multi-modal Agentic RAG for Enterprise Relations

- **POC Alignment:** Initiated daily standup cycle with the intern team and Kajal Ma'am.
- **Environment Setup:** Successfully replicated the required development environment on my personal laptop (Jan 22).
- **R&D & Learning:**
    - Reviewed provided resource materials.
        - Milvus vs FAISS
        - HIPAA (Privacy) and FHIR (Interoperability)
    - Explored Python libraries specific to our stack as well as brushing up on fundamentals.
    - Started exploring Document Loaders and RAG pipelines to understand data ingestion flows.

3. Strategic Initiative: Company-wide AI Upskilling

- **Course Design:** Collaborated on the curriculum design to equip all existing Aziro developers at all levels with AI capabilities.
- **Submission:** Submitted the "Courses for various roles and levels" proposal to Priyanka Ma'am (Jan 21).

Plan for Next Week

- Deepen understanding of the RAG pipeline architecture.
- Extract data using document loader from these:
    - .xlsx
    - .csv
    - .pptx
    - .docx
    - .url
    - A complex .pdf
- Can use custom scripts for logic if wanted (example summary using LLMs)

Week 2

Overview

This week focused on the practical execution of the RAG data ingestion pipeline and deep-diving into educational resources for the company-wide upskilling initiative. Key achievements included successful document parsing and curated resource selection from over 200 sources.

1. Administrative & Onboarding

- **Security Training**: Continued progress on the assigned Security Compliance training.

2. Project: Multi-modal Agentic RAG for Enterprise Relations

- **Data Extraction**: Successfully implemented and tested document loading using **PyMuPDF** and **Docling** (Jan 27) .
- **R&D & Learning**:
  - Initiated research into various **Chunking Strategies** (Fixed-size, Recursive, Semantic) to optimize data retrieval for the RAG pipeline (Jan 30).
  - Explored the nuances of handling complex PDFs via `Docling` to ensure high-fidelity data extraction.

3. Strategic Initiative: Company-wide AI Upskilling

- **Resource Curation**: Browsed through **200+ external resources** over a two-day sprint to identify world-class, free, and industry-relevant AI/ML training materials.
- **Submission**: Successfully compiled and submitted the "AI/ML Training External Links" document to Priyanka Ma'am before the deadline (Jan 29).

Adaptive document chunking

Adaptive chunking ensures that structured content, like code snippets, tables, or markdown documents, is preserved in a meaningful way. This prevents breaking important relationships within the data and improves retrieval efficiency.

Timestamp-based chunking

When dealing with conversational data from sources like Slack messages, system logs, or chat transcripts, traditional chunking methods often fail to capture the natural flow of interactions. Conversations are context-dependent, meaning the relevance of a message is often tied to previous messages and their timestamps.

Timestamp-based chunking groups messages based on specific time intervals (e.g., every 5

minutes, per session, or within an hour). This ensures that chunks retain contextual continuity, making them more effective for retrieval and analysis. ⍰

Week 3

Overview

Week 3 focused on the practical implementation of multimodal RAG capabilities and the launch of the internal upskilling initiative. Significant progress was made in image processing, adaptive chunking, and developing an automated HR recruitment tool using the Gemini API.

1. Administrative & Onboarding

- **Advanced Coursework**: Commenced the "Ramp up: Interns" coursework assigned by Ravi Sir to align technical skills with organizational standards (Feb 4).

2. Project: Multi-modal Agentic RAG for Enterprise Relations

- **Pipeline Optimization**:
  - Implemented a lightweight pipeline using **Docling**, **PyMuPDF**, and **Moondream/Qwen2-vl:3b** to ensure efficient performance on CPU-based environments (Feb 3).
  - Experiment with a lightweight embedding stack using **Ollama** (`sentence-transformer/nomic-embed-text`) to optimize the pipeline for resource-constrained devices (Feb 3).
- **Image Processing**: Successfully extracted ~90 images from complex documents, generating brief AI descriptions for each (Feb 3).
- **Adaptive Chunking**: Integrated the **HybridChunker** library from Docling for adaptive chunking, following guidance from Kajal Ma'am to improve retrieval precision (Feb 4).
- **Embedding Strategy**: Developed a workflow to store AI-generated image descriptions in lists for text-based chunking and embedding. Successfully embedded ~20 images using **embeddinggemma** (Feb 5).

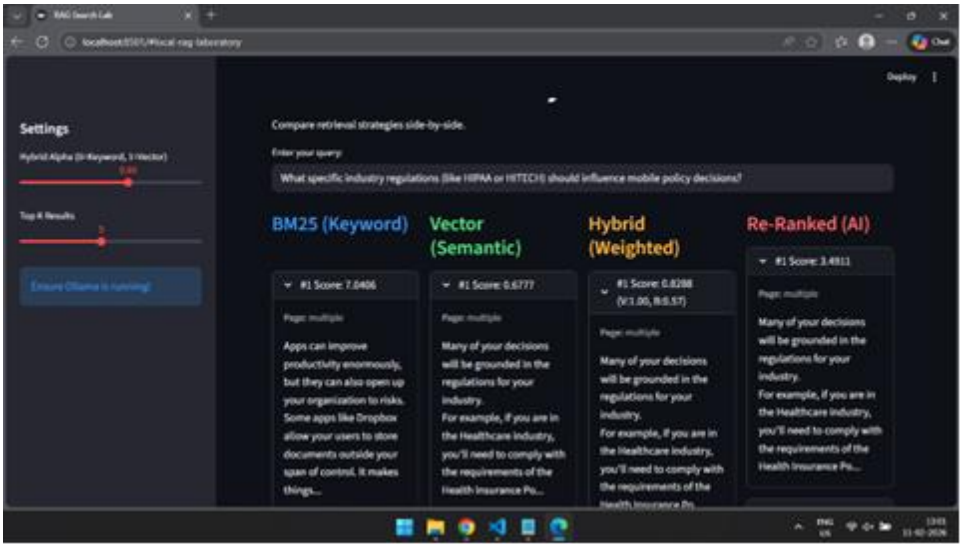
3. Strategic Initiative: Company-wide AI Upskilling

- **Program Launch**: Initiated the first "Upskilling Training" meeting (Feb 2).
- **Hiring automation Tool Demo**: given by Ravi sir and overlooked by Sagar sir.
  - **Features**: Gemini API-driven question generation based on domain/role, Streamlit frontend, automated Google Form creation, and bulk emailing capabilities for HR admins (Feb 2).

Plan for Next Week

- Started exploring **OpenAI's CLIP** model for advanced image processing and retrieval within the multimodal RAG framework (Feb 6).
- Refine the image-to-text-to-embedding pipeline to reduce processing time (currently 2+ hours for 20 images).
- Continue the "Ramp up: Interns" coursework.

Week 4



qdrant/fastembed: Fast, Accurate, Lightweight Python library to make State of the Art Embedding

Using Qdrant for embeddings search